



Interfaces Funcionales



Importante



Definición de prueba de Built-in Functional Interfaces

▼ Interfaces funcionales

▼ operaciones con enteros

- Realizar un conjunto de métodos (no estáticos) que realicen los siguientes cálculos:
 1. El cuadrado de un número
 2. El cubo de un número
 3. El doble de un número
 4. El triple de un número
 5. El resultado del polinomio

$$5 * x^3 + 7 * x^2 + 9$$

6. El resultado del polinomio variable

$$a * x^3 + b * x^2 + c$$

- Realizar el mismo problema con **Functional Interface**
- Resolver utilizando **Funcional interface** los siguientes problemas:
 1. A partir de un número entero, calcular:
 - a. La mitad
 - b. La cuarta parte
 - c. La décima parte
 - d. La n-sima parte
 2. A partir de un Array de enteros primitivos, calcular
 - a. La suma
 - b. La suma de los pares
 - c. La suma de los impares
 - d. La suma de los positivos
 - e. La suma de los negativos
 - f. La suma de los mayores de 100
 - g. La suma de los mayores de un número determinado

► **Solution**

▼ operaciones con String

- Realizar un conjunto de métodos (no estáticos) que comparen el String de la instancia con el String del parámetro para devolver true o false en función de:

1. Si son iguales

```
@FunctionalInterface interface MiInterfaceFuncional{
    boolean m(String s1, String s2); } public class
Ejemplo1 { public static void main(String ... args){
    MiInterfaceFuncional cadenasIguales = (s1, s2) ->
    s1.equals(s2); cadenasIguales.m("hola", "Hola"); //
    Devuelve false } }
```

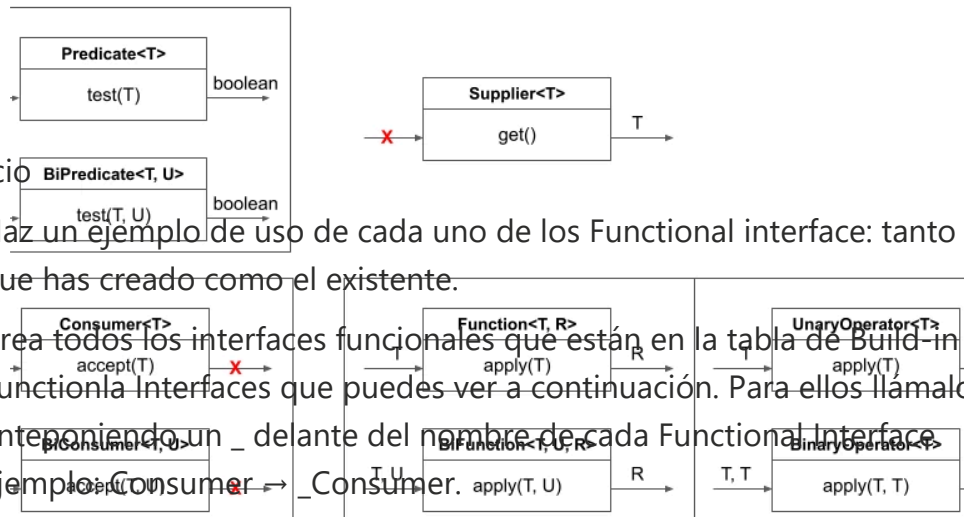
2. Si la instancia contiene al parámetro
 3. Si el número de caracteres es menor de 3.
 4. Si empieza por "A"
 5. Si termina por ".";
 6. Si la instancia empieza por un caracter determinado.
- Realizar el mismo problema con `Functional Interface`

▼ Built-in Functional Interfaces

Functional Interfaces	# Parameters	Return Type	Single Abstract Method
<code>Supplier<T></code>	0	T	<code>get</code>
<code>Consumer<T></code>	1 (T)	void	<code>accept</code>
<code>BiConsumer<T, U></code>	2 (T, U)	void	<code>accept</code>
<code>Predicate<T></code>	1 (T)	boolean	<code>test</code>
<code>BiPredicate<T, U></code>	2 (T, U)	boolean	<code>test</code>
<code>Function<T, R></code>	1 (T)	R	<code>apply</code>
<code>BiFunction<T, U, R></code>	2 (T, U)	R	<code>apply</code>
<code>UnaryOperator<T></code>	1 (T)	T	<code>apply</code>
<code>BinaryOperator<T></code>	2 (T, T)	T	<code>apply</code>

Common functional interfaces

Common Built-in Functional Interfaces



▼ Ejercicio

1. Haz un ejemplo de uso de cada uno de los Functional interface: tanto el que has creado como el existente.
2. Crea todos los interfaces funcionales que están en la tabla de Build-in Functionla Interfaces que puedes ver a continuación. Para ellos llámalos anteponiendo un `_` delante del nombre de cada Functional Interface. Ejemplo: `_Consumer`.

▼ Build-in FI

Functional Interfaces	# Parameters	Return Type	Single Abstract Method
<code>Supplier<T></code>	0	<code>T</code>	<code>get</code>
<code>Consumer<T></code>	1 (<code>T</code>)	<code>void</code>	<code>accept</code>
<code>BiConsumer<T, U></code>	2 (<code>T, U</code>)	<code>void</code>	<code>accept</code>
<code>Predicate<T></code>	1 (<code>T</code>)	<code>boolean</code>	<code>test</code>
<code>BiPredicate<T, U></code>	2 (<code>T, U</code>)	<code>boolean</code>	<code>test</code>
<code>Function<T, R></code>	1 (<code>T</code>)	<code>R</code>	<code>apply</code>
<code>BiFunction<T, U, R></code>	2 (<code>T, U</code>)	<code>R</code>	<code>apply</code>
<code>UnaryOperator<T></code>	1 (<code>T</code>)	<code>T</code>	<code>apply</code>
<code>BinaryOperator<T></code>	2 (<code>T, T</code>)	<code>T</code>	<code>apply</code>

▼ Simplificación de expresiones lambda